

แบบฝึกหัดกรณีศึกษา **กลุ่มที่ 2**

คำชี้แจง

ให้นักศึกษาเลือกวิธีแก้ปัญหาที่เหมาะสมสำหรับแต่ละกรณีศึกษา จากนั้นดำเนินการดังต่อไปนี้

1. ระบุข้อมูลนำเข้าและผลลัพธ์ที่ต้องการ
2. กำหนดความหมายของขนาดข้อมูล (n)
3. เขียนแนวคิดหรือ Pseudocode
4. เขียนโปรแกรมภาษา Java
5. ระบุคำสั่งหลัก หรือ Basic Operation ที่ใช้วิเคราะห์
6. วิเคราะห์จำนวนครั้งที่คำสั่งหลักทำงาน
7. วิเคราะห์กรณี Best Case, Worst Case และ Average Case หากสามารถวิเคราะห์ได้
8. ระบุ Time Complexity ในรูป Big O, Omega, Theta
9. วิเคราะห์ Auxiliary Space Complexity
10. เสนอแนวทางปรับปรุงอัลกอริทึมให้มีประสิทธิภาพมากขึ้น

กรณีศึกษาที่ 1: ระบบค้นหาห้สนักศึกษา

มหาวิทยาลัยจัดเก็บรหัสนักศึกษาไว้ในอาร์เรย์ เช่น

```
String[] studentIds = {  
    "6601001", "6601002", "6601003",  
    "6601004", "6601005"  
};
```

ระบบต้องรับรหัสนักศึกษาที่ต้องการค้นหา และแสดงตำแหน่งของรหัสดังกล่าว หากไม่พบให้แสดงข้อความว่า "Not found"

งานที่มอบหมาย

1. ออกแบบอัลกอริทึมค้นหาโดยไม่เปลี่ยนลำดับข้อมูล

Pseudocode

เริ่มต้น

รับรหัสนักศึกษาที่ต้องการค้นหา (target)

สำหรับ i = 0 ถึง studentIds.length - 1

 ถ้า studentIds[i] เท่ากับ target

 แสดงตำแหน่ง i

จบการทำงาน

จบเงื่อนไข

จบการวนซ้ำ

แสดง "Not found"

จบ

2. เขียนเมธอด Java ชื่อ

```
static int searchStudent(String[] studentIds, String target)
static int searchStudent(String[] studentIds, String target) {

    for (int i = 0; i < studentIds.length; i++) {
        if (studentIds[i].equals(target)) {
            return i;
        }
    }

    return -1;
}
```

3. กำหนดให้ (n) หมายถึงอะไร

ตอบ หมายถึง จำนวนรหัสนักศึกษาทั้งหมด ที่เก็บอยู่ใน studentIds

4. วิเคราะห์กรณีต่อไปนี้

- รหัสอยู่ตำแหน่งแรก
- รหัสอยู่ตำแหน่งสุดท้าย
- ไม่มีรหัสที่ค้นหา

กรณีที่ 1 รหัสอยู่ตำแหน่งแรก

เช่น ค้นหา "6601001"

- เปรียบเทียบ 1 ครั้งก็เจอ
- ใช้เวลาน้อยที่สุด

Time = $\Omega(1)$

กรณีที่ 2 รหัสอยู่ตำแหน่งสุดท้าย

เช่น "6601005"

- ต้องตรวจทุกตัว
- เปรียบเทียบทั้งหมด n ครั้ง

Time = $O(n)$

กรณีที่ 3 ไม่มีรหัสที่ค้นหา

เช่น "6601010"

- ต้องตรวจสอบทุกตัว
- ไม่พบข้อมูล

Time = $O(n)$

5. ระบุ Time Complexity ในรูป Big O, Omega, Theta

Best Case

$\Omega(1)$

ค้นหาเจอตั้งแต่ตำแหน่งแรก เปรียบเทียบเพียง 1 ครั้ง

Average Case

$O(n)$

โดยเฉลี่ยต้องตรวจสอบประมาณครึ่งหนึ่งของข้อมูล จึงเติบโตเป็นเชิงเส้น

Worst Case

$O(n)$

ค้นหาจนถึงตัวสุดท้ายหรือไม่พบข้อมูล ต้องตรวจสอบ n ตัว

6. หากข้อมูลถูกเรียงลำดับแล้ว นักศึกษาจะเลือกอัลกอริทึมใดเพื่อให้ค้นหาเร็วขึ้น

ตอบ Binary Search

คำถามวิเคราะห์

1. คำสั่งเปรียบเทียบทำงานอย่างน้อยกี่ครั้ง

ตอบ 1 ครั้ง

2. คำสั่งเปรียบเทียบทำงานมากที่สุดกี่ครั้ง

ตอบ 5 ครั้ง ถ้าหา 6601005

3. ข้อมูลจำนวนมากส่งผลต่อเวลาการค้นหอย่างไร

ตอบ ถ้าจำนวนข้อมูล n เพิ่มขึ้น เวลาที่ใช้ในการค้นหาก็เพิ่มขึ้นตาม เพราะ Linear Search

ต้องตรวจสอบข้อมูลที่ละตัว